

Caso único La planta inteligente — 20 puntos

Una empresa agroindustrial desarrolla un sistema de visión por computadora para clasificar hojas de plantas en tres estados: sana, con plaga, con hongo. Las imágenes son en escala de grises de 32×32 píxeles. El equipo dispone de 1 000 imágenes etiquetadas (333 por categoría). Su primera red, entrenada sin funciones de activación, no mejora aunque agregan más capas. Deben decidir qué arquitectura usar y cómo configurar la red.

Instrucciones: justifique todas sus respuestas con argumentos técnicos del curso. No basta con mencionar el concepto: explique por qué aplica al caso.

Pregunta 1 — Kernels y feature maps (7 pts)

1.a (2 pts) Filtros clásicos vs filtros aprendidos

El equipo evalúa preprocesar las imágenes con un filtro Sobel antes de pasarlas a la CNN.

¿Qué diferencia fundamental existe entre un filtro Sobel y un filtro que aprende la CNN? Explique por qué, para detectar manchas de plaga o hongo en hojas, los filtros aprendidos son más apropiados que el Sobel.

1.b (2 pts) Feature maps en paralelo

Feature map: resultado de aplicar un filtro sobre una imagen. Cada filtro produce un mapa que resalta las zonas donde su patrón está presente. Una CNN usa decenas de filtros en paralelo, produciendo así decenas de feature maps distintos de la misma imagen, cada uno capturando información diferente.

La primera capa Conv del sistema usa 32 filtros sobre la misma imagen de hoja. Explique por qué obtener 32 representaciones distintas de la misma imagen es necesario para distinguir entre plaga y hongo. ¿Qué pasaría si la capa usara un solo filtro?

1.c (3 pts) Jerarquía de capas aplicada al problema

Jerarquía de características: las capas de una CNN no detectan todas lo mismo.

Capa 1: bordes y gradientes simples. Capa 2: texturas y formas. Capa 3+: conceptos de alto nivel.

Cada capa opera sobre el feature map de la anterior, no sobre la imagen original.

El sistema tiene tres capas convolucionales. Explique qué tipo de características detectaría cada capa en las imágenes de hojas, siendo específico para este problema: no repita la explicación general, aplíquela a hojas sanas, con plaga y con hongo.

Luego argumente: ¿por qué es importante que la red tenga al menos tres capas para este problema y no solo una?

Pregunta 1

- a. La diferencia fundamental es que el filtro Sobel tiene coeficientes fijos definidos manualmente, mientras que los filtros de una CNN son aprendidos automáticamente durante el entrenamiento mediante backpropagation.

El filtro de Sobel es más que todo para detectar cambios en la intensidad como bordes y gradientes en una imagen. Las máscaras que usa son siempre las mismas (ya están definidas).

Mientras que CNN ajusta los valores de los kernels utilizando las imágenes etiquetadas del dataset de entrenamiento. Durante el entrenamiento cada filtro va especializándose en detectar patrones que ayuden lo más posible para distinguir las clases.

Tomando en cuenta el caso dado, para clasificar hojas en los estados de sano, con plaga y con hongo... los filtros aprendidos serían los más apropiados porque las diferencias entre las clases no dependen únicamente de los bordes sino de más características que varían en las diferentes clases. Por ejemplo, las plagas pueden generar pequeñas manchas, perforaciones, mientras que los hongos suelen producir texturas o patrones sobre la superficie de la hoja.

Un filtro Sobel resaltaría únicamente los contornos de estas regiones, pero no aprendería automáticamente las características particulares que distinguen una plaga de un hongo.

- b. Eso es porque cada filtro de una CNN busca un patrón diferente dentro de la imagen. Al aplicar 32 filtros en paralelo, la red genera 32 feature distintos, donde cada uno resalta una característica específica de la hoja.

De esta forma, las diferencias entre plaga y hongo ya pueden ser más obvios para la red y puede distinguir de mejor manera.

En cambio, un único filtro no puede representar adecuadamente estas diferencias entre las dos clases. Se produciría una sola feature map, lo que limita de gran manera la capacidad de la red de detectar patrones complejos.

- c. En una CNN, cada capa aprende características de complejidad creciente.

Primera Capa: La primera capa trabaja directamente sobre la imagen original y detectaría cosas básicas como:

- bordes de la hoja
- cambios de intensidad
- manchas

Segunda Capa: La segunda capa recibe los feature maps de la primera capa y combina los bordes detectados para formar patrones más complejos como:

- texturas de hongos
- agrupaciones de manchas
- regiones de la hoja perforadas (signo de plagas)
- formas repetitivas sobre la hoja

Tercera Capa: La tercera capa combina las texturas y formas detectadas previamente para construir representaciones de alto nivel. Puede aprender conceptos como:

- Patron global comun de una hoja sana
- Características únicas de lesiones causadas por plagas
- Características especiales típicas de infecciones por hongos

¿Por qué son necesarias al menos tres capas y no una sola?

Con una única capa convolucional la red solamente podría detectar características simples, en cambio con más podría detectar patrones más complejos. Y eso es lo ideal ya que la diferencia entre una hoja sana, una hoja con plaga y una hoja con hongo depende de patrones más complejos que involucran texturas, formas entre regiones afectadas.

Pregunta 2 — Red neuronal y arquitectura (13 pts)

2.a (2 pts) El problema de la linealidad

La primera red del equipo usa solo operaciones lineales entre capas, sin ninguna función de activación.

Explique por qué una red con solo operaciones lineales no puede aprender a distinguir manchas de plaga de manchas de hongo, sin importar cuántas capas tenga. Use el concepto de linealidad en su respuesta.

2.b (3 pts) ReLU y su efecto en el aprendizaje

ReLU $f(x) = \max(0, x)$: activa la neurona solo si su valor es positivo. Elimina activaciones negativas. Introduce no-linealidad, lo que permite a la red aprender representaciones complejas. Una neurona con salida 0 se dice "inactiva": no transmite información a la siguiente capa.

El equipo agrega ReLU después de cada capa Conv. Explique cómo ReLU contribuye a que la red aprenda a diferenciar una hoja sana de una infectada. ¿Qué información se perdería si en vez de ReLU usaran la función identidad $f(x) = x$? Además: el modelo sigue fallando con manchas de color muy oscuro sobre hojas oscuras. Un compañero sugiere que ReLU podría ser parte del problema. ¿Tiene sentido esa hipótesis? Justifique.

2.c (4 pts) Elección de arquitectura: LeNet-5 vs AlexNet

LeNet-5 (1998): red pequeña, 5 capas. Entrada 32×32 escala de grises. Sin preentrenamiento en ImageNet. Diseñada para patrones simples. Funciona bien con datasets pequeños.
AlexNet (2012): red profunda, 8 capas. Entrada 224×224 a color. Preentrenada en ImageNet (1M+ imágenes). Transfer learning: congelar las capas tempranas (características genéricas ya aprendidas) y reentrenar solo las capas finales con los nuevos datos.
CIFAR-10: benchmark de referencia con 60 000 imágenes 32×32 en 10 clases. No es un modelo, sino un conjunto de datos estándar para comparar el rendimiento de distintas arquitecturas.

Con 1 000 imágenes etiquetadas e imágenes de 32×32 en escala de grises, el equipo debe elegir entre entrenar LeNet-5 desde cero o usar AlexNet con transfer learning.

Analice ambas opciones considerando: tamaño del dataset, compatibilidad de las imágenes y riesgo de sobreajuste. Elija una, justifique su decisión e identifique el riesgo concreto de la opción que descarta.

2.d (4 pts) Decisión de diseño global

Tomando todo lo discutido en esta práctica, describa cómo configuraría el sistema completo de clasificación de hojas: qué tipo de filtros usaría en las primeras capas, cuántas capas convolucionales tendría la red, qué función de activación elegiría y qué arquitectura base seleccionaría.

No basta con listar las decisiones: explique cómo cada una responde a una característica concreta del problema (tipo de imagen, cantidad de datos, tipo de patrón a detectar).

Pregunta 2

- a. Una red formada únicamente por operaciones lineales no puede aprender relaciones complejas entre los datos porque la composición de funciones lineales sigue siendo una función lineal.

Si una capa realiza una transformación como: $y = W1 * x + b1$ y la siguiente: $z = W2*y + b2$

Ambas pueden combinarse en una sola transformación lineal. Entonces, agregar mas capas lineales no necesariamente aumenta la capacidad de representación de la red.

En el problema que tenemos, la separación entre las clases sana, con plaga o cn hongo depende más de patrones complejos como mencione antes. Estas características generan relaciones no lineales en el espacio de datos.

Una red exclusivamente lineal solo puede construir fronteras de decisión lineales, por lo que no tiene capacidad suficiente para modelar la complejidad visual necesaria para distinguir entre manchas de plaga y manchas de hongo

- b. La función ReLU introduce no linealidad en la red neuronal, lo que permite que las distintas capas construyan representaciones cada vez más complejas de la imagen.

En la detección de hojas, las primeras capas detectan bordes, manchas o texturas. Gracias a ReLU las activaciones que representan características relevantes se conservan, mientras que aquellas negativas se eliminan (se reemplazan con cero).

Esto permite que las capas superiores combinen únicamente las mejores respuestas para reconocer patrones asociadas a cada clase.

En cambio, si se utilizara una función identidad, esta no introduciría ninguna no linealidad. Lo que causaría que toda la red volvería a ser equivalente a una única transformación lineal. Se perdería la capacidad de aprender patrones complejos necesarios para la clasificación.

Sobre la hipótesis de que ReLU puede ser parte del problema... es cierto pero no del todo.

ReLU elimina todas las activaciones negativas. Por ejemplo, si una mancha muy oscura sobre una hoja oscura genera respuestas débiles o negativas en algunos filtros, entonces esas respuestas podrían convertirse en cero y dejar de propagarse por la red.

Eso podría provocar pérdida de información en regiones de contraste bajo, donde las diferencias entre la mancha y el fondo son más pequeñas.

Sin embargo, el problema no es únicamente de ReLU, también podrían influir:

- Bajo contraste de la imagen
- Resolución reducida
- Cantidad limitada de datos
- Filtros que no han aprendido lo suficiente sobre características.

- c. Personalmente yo elegiría LeNet-5. La principal razón es que la arquitectura fue diseñada para imágenes específicamente de tamaño similar y funciona adecuadamente con conjuntos de datos algo pequeños. Además, su menor número de parámetros reduce el riesgo de memorizar el dataset.

Ahora, el porqué no utilizaría AlexNet, es por el overfitting. Esta red posee una cantidad mucho mayor de parámetros que LeNet y fue diseñada para problemas mucho más grandes. Contamos con solo 1000 imágenes, eso significa que un AlexNet podría adaptarse de manera excesiva a los datos del dataset y generalizar mal sobre nuevas hojas no vistas.

- d. Para la decisión del diseño final, yo configuraría el siguiente workflow:
1. Filtros en las primeras capas: Utilizaría filtros que aprenden por la CNN en vez de fijos como Sobel. Eso permite que los filtros puedan especializarse en detectar patrones complejos en cada clase de las hojas. Por ejemplo:
 - texturas de hongos,
 - bordes de lesiones,
 - perforaciones por plagas, etc.Como resultado, la red extrae características mucho más discriminantes que con las obtenidas con filtros estáticos.
 2. Nro de capas convolucionales: Al menos 3 capas convolucionales. Con la primera capa para detección de bordes y gradientes locales, segunda capa

para detección de texturas y grupos de manchas. Eso es necesario ya que las diferencias entre clases no dependen de un único borde sino de combinaciones de características.

3. La función de activación que usaría sería ReLU. Esta introduce la no linealidad que es necesaria para que la red neuronal aprenda representaciones complejas y evita que diferentes capas se comporten como una sola transformación lineal.
4. Arquitectura base: Como mencioné antes, yo me iría por LeNet-5 como modelo de arquitectura base. Las razones detrás de esto son:
 - Las imágenes son todas de 32x32 píxeles
 - Las imágenes están en escala de grises
 - El dataset es algo pequeño (1000 ejemplos etiquetados=
 - El riesgo de overfitting debe mantenerse bajo, lo que no sucede en AlexNet.

LeNet-5 fue diseñada precisamente para imágenes pequeñas y requiere menos datos para entrenarse de manera efectiva.